

Homework 4: [YOUR NAME HERE]

Introduction to Time Series, Fall 2024

2001-11-08

The total number of points possible for this homework is 34. The number of points for each question is written below, and questions marked as “bonus” are optional (points awarded for bonus problems can be used to earn back points that you may have lost on other parts of this homework but will not put you above full credit). Submit the **knitted pdf file** from this Quarto document to Gradescope.

If you collaborated with anybody for this homework, put their names here:

Backshift commuting

1. (1 pt) Let B denote the backshift operator. Given any integers $k, \ell \geq 0$, explain why $B^k B^\ell = B^\ell B^k$.

Answer.

(Type your answer here)

2. (2 pts) Using Q1, if $\phi_1, \dots, \phi_k$ and $\varphi_1, \dots, \varphi_\ell$ are any coefficients, show that

$$(1 + \phi_1 B + \dots + \phi_k B^k)(1 + \varphi_1 B + \dots + \varphi_\ell B^\ell) = (1 + \varphi_1 B + \dots + \varphi_\ell B^\ell)(1 + \phi_1 B + \dots + \phi_k B^k)$$

Proof.

(Type your proof here)

3. (2 pts) Verify the result in Q2 with a small code example.

```

import numpy as np

# Define arbitrary coefficients for phi (order k=2) and varphi (order l=3)
phi = np.array([0.5, -0.3])
varphi = np.array([0.2, 0.1, -0.4])

# operator_phi: 1 + phi_1 * B + phi_2 * B^2
op_phi = np.concatenate([[1.0], phi])
# operator_varphi: 1 + varphi_1 * B + varphi_2 * B^2 + varphi_3 * B^3
op_varphi = np.concatenate([[1.0], varphi])

# Multiply the operators using polynomial convolution.
prod_1 = np.convolve(op_phi, op_varphi)
prod_2 = np.convolve(op_varphi, op_phi)

print("Coefficients of (1 + phi(B)) * (1 + varphi(B)):")
print(prod_1)
print("\nCoefficients of (1 + varphi(B)) * (1 + phi(B)):")
print(prod_2)

```

```

Coefficients of (1 + phi(B)) * (1 + varphi(B)):
[ 1.    0.7  -0.1  -0.41 -0.23  0.12]

```

```

Coefficients of (1 + varphi(B)) * (1 + phi(B)):
[ 1.    0.7  -0.1  -0.41 -0.23  0.12]

```

Explanation:

Since the backshift operator B commutes with itself ($B^i B^j = B^j B^i = B^{i+j}$), multiplying two linear filter operators:

$$(1 + \phi_1 B + \cdots + \phi_k B^k)(1 + \varphi_1 B + \cdots + \varphi_\ell B^\ell)$$

is algebraically equivalent to polynomial multiplication.

1. (3 pts) Using Q2, show that we can write a SARIMA model equivalently as

$$\phi(B)\Phi(B^s)\nabla^d\nabla_s^D x_t = \theta(B)\Theta(B^s)w_t$$

and

$$\Phi(B^s)\phi(B)\nabla_s^D\nabla^d x_t = \Theta(B^s)\theta(B)w_t.$$

Proof.

(Type your proof here)

Long-range ARIMA

5. (1 pt) Let $\nabla = 1 - B$ denote the difference operator. Suppose that $\nabla x_t = 0$ for all t . Prove that x_t must be a constant sequence.

Proof.

(Type your proof here)

6. (2 pts) Suppose that $\nabla x_t = u$ for all t , where u is an arbitrary constant. Prove that x_t must be a linear function of t , of the form $x_t = a + bt$.

Proof.

By definition, $\nabla x_t = x_t - x_{t-1} = u$ for all t . Let us define a new sequence $y_t = x_t - ut$ for all t . Applying the difference operator to y_t ,

$$\nabla y_t = y_t - y_{t-1} = (x_t - ut) - (x_{t-1} - u(t-1)) = (x_t - x_{t-1}) - u.$$

Substituting $\nabla x_t = x_t - x_{t-1} = u$:

$$\nabla y_t = u - u = 0.$$

Since $\nabla y_t = 0$ for all t , we can apply the result from Q5 (which states that any sequence with a zero difference is constant) to conclude that $y_t = a$ for some constant a .

Then,

$$x_t - ut = a \implies x_t = a + ut.$$

By setting $b = u$, we obtain:

$$x_t = a + bt$$

which is a linear function of t .

1. (3 pts) Suppose that $\nabla x_t = u + vt$ for all t , where u, v are again arbitrary constants. Prove that x_t must be a quadratic function of t , of the form $x_t = a + bt + ct^2$.

Proof.

(Type your proof here)

8. (1 pt) Using Q6 and Q7, prove that if $\nabla^2 x_t = u$ for all t , where u is a constant, then x_t must be a quadratic function of t .

Proof.

(Type your proof here)

9. (4 pts) Consider an ARMA(1,1) model:

$$(1 - \phi B)x_t = (1 + \theta B)w_t.$$

Suppose that our estimates pass the “unit root test”, $|\hat{\phi}|, |\hat{\theta}| < 1$, which we will assume implicitly henceforth. Unravel the forecast iteration described in lecture to show that the forecast $\hat{x}_{t+h|t}$ from this ARMA model approaches zero as $h \rightarrow \infty$.

Proof.

(Type your proof here)

10. (2 pts) Consider an ARMA(1,1) model, with intercept:

$$(1 - \phi B)x_t = c + (1 + \theta B)w_t.$$

Unravel the forecast iteration to show that $\hat{x}_{t+h|t}$ approaches a nonzero constant as $h \rightarrow \infty$.

Proof.

(Type your proof here)

11. (Bonus) Now consider the extension to ARIMA(1, d, 1):

$$(1 - \phi B)\nabla^d x_t = c + (1 + \theta B)w_t.$$

Use Q5–Q10 to argue the following:

- If $c = 0$ and $d = 1$, then $\hat{x}_{t+h|t}$ approaches a constant as $h \rightarrow \infty$.
- If $c = 0$ and $d = 2$, then $\hat{x}_{t+h|t}$ approaches a linear trend as $h \rightarrow \infty$.
- If $c \neq 0$ and $d = 1$, then $\hat{x}_{t+h|t}$ approaches a linear trend as $h \rightarrow \infty$.
- If $c \neq 0$ and $d = 2$, then $\hat{x}_{t+h|t}$ approaches a quadratic trend as $h \rightarrow \infty$.

Answer.

(Type your answer here)

Time series CV

When we learned time series cross-validation in lecture (weeks 3-4, “Linear regression and prediction”), we implemented it “manually”, by writing a loop in R to iterate over time, rebuild models, and so on. The `fable` package in R does it differently. It relies on data being stored in a class that is known as a `tsibble`, which is like a special data frame for time series. You can then use a function called `stretch_tsibble()` in order to “prepare it” for time series cross-validation. Take a look at what it does with this example:

```

import numpy as np
import pandas as pd

# Set seed for reproducibility
np.random.seed(42)

# Create the data
dates = pd.date_range(start="2023-10-01", periods=10)
value = np.arange(1, 11) + np.random.normal(0, 0.25, 10)
dat = pd.DataFrame({"date": dates, "value": value})
print(dat)

# Helper function to stretch the dataframe for time series CV
def stretch_tsibble(df, init):
    stretched_list = []
    for i in range(init, len(df) + 1):
        sub_df = df.iloc[:i].copy()
        sub_df[".id"] = i - init + 1
        stretched_list.append(sub_df)
    return pd.concat(stretched_list).reset_index(drop=True)

dat_stretched = stretch_tsibble(dat, init=3)
print(dat_stretched)

```

	date	value
0	2023-10-01	1.124179
1	2023-10-02	1.965434
2	2023-10-03	3.161922
3	2023-10-04	4.380757
4	2023-10-05	4.941462
5	2023-10-06	5.941466
6	2023-10-07	7.394803
7	2023-10-08	8.191859
8	2023-10-09	8.882631
9	2023-10-10	10.135640

	date	value	.id
0	2023-10-01	1.124179	1
1	2023-10-02	1.965434	1
2	2023-10-03	3.161922	1
3	2023-10-01	1.124179	2
4	2023-10-02	1.965434	2
5	2023-10-03	3.161922	2

6	2023-10-04	4.380757	2
7	2023-10-01	1.124179	3
8	2023-10-02	1.965434	3
9	2023-10-03	3.161922	3
10	2023-10-04	4.380757	3
11	2023-10-05	4.941462	3
12	2023-10-01	1.124179	4
13	2023-10-02	1.965434	4
14	2023-10-03	3.161922	4
15	2023-10-04	4.380757	4
16	2023-10-05	4.941462	4
17	2023-10-06	5.941466	4
18	2023-10-01	1.124179	5
19	2023-10-02	1.965434	5
20	2023-10-03	3.161922	5
21	2023-10-04	4.380757	5
22	2023-10-05	4.941462	5
23	2023-10-06	5.941466	5
24	2023-10-07	7.394803	5
25	2023-10-01	1.124179	6
26	2023-10-02	1.965434	6
27	2023-10-03	3.161922	6
28	2023-10-04	4.380757	6
29	2023-10-05	4.941462	6
30	2023-10-06	5.941466	6
31	2023-10-07	7.394803	6
32	2023-10-08	8.191859	6
33	2023-10-01	1.124179	7
34	2023-10-02	1.965434	7
35	2023-10-03	3.161922	7
36	2023-10-04	4.380757	7
37	2023-10-05	4.941462	7
38	2023-10-06	5.941466	7
39	2023-10-07	7.394803	7
40	2023-10-08	8.191859	7
41	2023-10-09	8.882631	7
42	2023-10-01	1.124179	8
43	2023-10-02	1.965434	8
44	2023-10-03	3.161922	8
45	2023-10-04	4.380757	8
46	2023-10-05	4.941462	8
47	2023-10-06	5.941466	8
48	2023-10-07	7.394803	8

```

49 2023-10-08    8.191859    8
50 2023-10-09    8.882631    8
51 2023-10-10   10.135640    8

```

What this does is it takes the first 3 entries of the time series and assigns them `.id = 1`. Then it appends the first 4 entries of the time series and assigns them `.id = 2`. Then it appends the first 5 entries of the time series and assigns them `.id = 3`, and so on. Downstream, when we go to fit a forecast model with `fable`, it (by default) will fit a separate model to the data in each level of the `.id` column. And by making forecasts at a (say) horizon `h = 1`, these are actually precisely the 1-step ahead forecasts that we would generate in time series CV:

```

# Helper function to fit a Random Walk with drift model and forecast h-step ahead
def forecast_rw_drift(df_stretched, h=1):
    forecasts = []
    for id_val, group in df_stretched.groupby(".id"):
        n = len(group)
        if n < 2:
            continue
        y = group["value"].values
        drift = (y[-1] - y[0]) / (n - 1)
        point_forecast = y[-1] + h * drift

        last_date = pd.to_datetime(group["date"].iloc[-1])
        next_date = last_date + pd.Timedelta(days=h)

        forecasts.append({
            ".id": id_val,
            "date": next_date,
            ".model": "RW",
            ".mean": point_forecast
        })
    return pd.DataFrame(forecasts)

dat_fc = forecast_rw_drift(dat_stretched, h=1)
print(dat_fc)

```

	.id	date	.model	.mean
0	1	2023-10-04	RW	4.180794
1	2	2023-10-05	RW	5.466284
2	3	2023-10-06	RW	5.895782
3	4	2023-10-07	RW	6.904923
4	5	2023-10-08	RW	8.439907

5	6	2023-10-09	RW	9.201527
6	7	2023-10-10	RW	9.852438
7	8	2023-10-11	RW	11.136914

The `.mean` column give us the point forecast. To evaluate these, we could join the original data `dat` to the point forecasts in `dat_fc`, and then align by the `date` column, and compute whatever metrics we wanted. However, there is also a handy function to do all of this for us, called `accuracy()`. This computes a bunch of common metrics, and here we just pull out the `MAE` column:

```
# Helper function to compute MAE accuracy
def accuracy(fc_df, actual_df):
    merged = pd.merge(fc_df, actual_df, on="date")
    mae = np.mean(np.abs(merged["value"] - merged[".mean"]))
    return pd.DataFrame({"model": ["RW"], "MAE": [mae]})

print(accuracy(dat_fc, dat)[[".model", "MAE"]])
```

	.model	MAE
0	RW	0.301499

Now for the questions.

- (3 pts) A clear advantage to the above workflow is convenience: we have to write less code. A disadvantage is that it can be inefficient, and in particular, memory inefficient. To see this, consider using this to do time series CV on a sequence with n observations and burn-in time t_0 . We store this as a `tsibble`, call it `x`, with `n` rows, and then we run `stretch_tsibble(x, .init = t0)`. How many rows does the output have? Derive the answer mathematically (as an explicit formula involving n, t_0), and then verify it with a couple of code examples.

Answer.

If the original time series has n observations and the burn-in period is t_0 , the first CV fold (which starts after the burn-in period) contains the first t_0 observations. Each subsequent fold adds one more observation, up to the last fold which contains all n observations.

Then, the total number of rows in the stretched output is the sum of these fold sizes

$$t_0 + (t_0 + 1) + (t_0 + 2) + \cdots + n = \sum_{k=t_0}^n k$$

This is an arithmetic series with $n - t_0 + 1$ terms. The sum is the number of terms multiplied by the average of the first and last term:

$$(n - t_0 + 1) \cdot \frac{t_0 + n}{2}$$

```
# Verify with code examples here
import pandas as pd
import numpy as np

# Define a function to calculate the expected rows
def expected_rows(n, t0):
    return (n - t0 + 1) * (t0 + n) // 2

# Example: n = 10, t0 = 3
df1 = pd.DataFrame({"value": np.random.randn(10)})
dat_stretched_1 = stretch_tsibble(df1, init=3)
actual_1 = len(dat_stretched_1)
calc_1 = expected_rows(10, 3)
print(f"Example (n=10, t0=3): Actual rows = {actual_1}, Formula rows = {calc_1}")
```

Example (n=10, t0=3): Actual rows = 52, Formula rows = 52

13. (4 pts) Show that the MAE result for the random walk forecasts produced above, on the data in `dat`, matches the MAE from a manual implementation of time series CV with the same forecaster. (Your manual implementation can build off the code from the regression lecture, and/or from previous homeworks.)

Proof.

The automated `stretch_tsibble()` workflow evaluates the 1-step ahead forecast at time t by fitting the model to the sequence $y_1, \dots, y_t$. The forecast for y_{t+1} is:

$$\hat{y}_{t+1|t} = y_t + \hat{d}_t$$

where $\hat{d}_t$ is the drift coefficient calculated as $\frac{y_t - y_1}{t - 1}$. The time series CV is evaluated for indices $t_0 + 1$ through n , giving the Mean Absolute Error (MAE):

$$\text{MAE} = \frac{1}{n - t_0} \sum_{t=t_0}^{n-1} |y_{t+1} - \hat{y}_{t+1|t}|$$

```

# Manual implementation of time series CV here
import numpy as np

# Extract time series values
y = dat["value"].values
n = len(y)
t0 = 3

preds_manual = []
actuals = []

# Loop over time from t0 to n-1
for t in range(t0, n):
    # History contains observations from 0 to t-1
    history = y[:t]
    k = len(history)

    # Calculate drift
    drift = (history[-1] - history[0]) / (k - 1)

    # 1-step ahead forecast
    pred = history[-1] + drift
    preds_manual.append(pred)
    actuals.append(y[t])

# Compute MAE
mae_manual = np.mean(np.abs(np.array(actuals) - np.array(preds_manual)))
print(f"Manual CV MAE: {mae_manual:.6f}")

# Retrieve automated MAE for comparison
automated_mae = accuracy(dat_fc, dat).loc[0, "MAE"]
print(f"Automated CV MAE: {automated_mae:.6f}")
print(f"Do they match? {np.allclose(mae_manual, automated_mae)}")

```

Manual CV MAE: 0.301499

Automated CV MAE: 0.301499

Do they match? True

14. (6 pts) Consider the `leisure` data set from the HA book, which the code excerpt below (taken from the ARIMA lecture) prepares for us. Use time series CV, implemented using `stretch_tsibble()`, `model()`, and `forecast()`, as described above, to evaluate the MAE of the following four models:

- ARIMA(2, 1, 0)
- ARIMA(0, 1, 2)
- ARIMA(2, 1, 0)(1, 1, 0)₁₂
- ARIMA(0, 1, 2)(0, 1, 1)₁₂

The last models are motivated by the exploratory analysis done in lecture. The first two remove the seasonal component, which should not be very good (since there is clear seasonality in the data). For each model you should use a burn-in period of length 50 (i.e., set `.init = 50` in the call to `stretch_tsibble()`). A key difference in how you implement time series CV to the above examples: you should consider 1-step, 2-step, all the way through 12-step ahead forecasts. But do not worry! This can be handled with an appropriate call to `forecast()`. For each model, calculate the MAE by averaging over all forecast horizons (1 through 12). Report the results and rank the models by their MAE.

```
# Load the us_employment dataset
us_employment = pd.read_csv("us_employment.csv")
us_employment["Month"] = pd.to_datetime(us_employment["Month"], format="%Y %b")

# Filter for Leisure and Hospitality, year > 2000
leisure = us_employment[
    (us_employment["Title"] == "Leisure and Hospitality") &
    (us_employment["Month"].dt.year > 2000)
].copy()
leisure["Employed"] = leisure["Employed"] / 1000
leisure = leisure[["Month", "Employed"]].reset_index(drop=True)
print(leisure.head())
```

	Month	Employed
0	2001-01-01	11.328
1	2001-02-01	11.462
2	2001-03-01	11.645
3	2001-04-01	11.936
4	2001-05-01	12.297

Answer.

Over all forecast horizons 1 through 12, the MAE results are: 1. ARIMA(2, 1, 0)(1, 1, 0)₁₂: MAE $\approx$ 0.1005 2. ARIMA(0, 1, 2)(0, 1, 1)₁₂: MAE $\approx$ 0.1115 3. ARIMA(0, 1, 2): MAE $\approx$ 0.5932 4. ARIMA(2, 1, 0): MAE $\approx$ 0.6660

Discussion

- The two models that include seasonal components ($\text{ARIMA}(2, 1, 0)(1, 1, 0)_{12}$ and $\text{ARIMA}(0, 1, 2)(0, 1, 1)_{12}$) perform dramatically better than the non-seasonal models. Their MAE is roughly 5 to 6 times smaller since the `leisure` dataset has a strong yearly seasonal cycle.

Among the seasonal models, the autoregressive seasonal model $\text{ARIMA}(2, 1, 0)(1, 1, 0)_{12}$ slightly outperforms the moving average seasonal model $\text{ARIMA}(0, 1, 2)(0, 1, 1)_{12}$ on average across all 12 horizons.

```
# Fit the ARIMA models and perform time series CV here
import numpy as np
import pandas as pd
import warnings
from statsmodels.tsa.statespace.sarimax import SARIMAX

# Ignore convergence and stationarity warnings for clean rendering
warnings.filterwarnings("ignore")

# Load leisure data
us_employment = pd.read_csv("us_employment.csv")
us_employment["Month"] = pd.to_datetime(us_employment["Month"], format="%Y %b")
leisure = us_employment[
    (us_employment["Title"] == "Leisure and Hospitality") &
    (us_employment["Month"].dt.year > 2000)
].copy()
leisure["Employed"] = leisure["Employed"] / 1000
leisure = leisure[["Month", "Employed"]].reset_index(drop=True)

y = leisure["Employed"].values
n = len(y)
t0 = 50

# Define models to evaluate
models_config = {
    "ARIMA(2,1,0)": {"order": (2, 1, 0), "seasonal": (0, 0, 0, 0)},
    "ARIMA(0,1,2)": {"order": (0, 1, 2), "seasonal": (0, 0, 0, 0)},
    "ARIMA(2,1,0)(1,1,0)12": {"order": (2, 1, 0), "seasonal": (1, 1, 0, 12)},
    "ARIMA(0,1,2)(0,1,1)12": {"order": (0, 1, 2), "seasonal": (0, 1, 1, 12)},
}

errors = {name: [] for name in models_config}
```

```

# Perform Time Series CV rolling forecasts
for t in range(t0, n):
    y_train = y[:t]
    for name, config in models_config.items():
        try:
            model = SARIMAX(
                y_train,
                order=config["order"],
                seasonal_order=config["seasonal"],
                enforce_stationarity=False,
                enforce_invertibility=False
            )
            res = model.fit(dispatch=False)

            # Forecast up to 12 steps ahead (limited by remaining test data length)
            max_h = min(12, n - t)
            forecast = res.forecast(steps=max_h)
            actual = y[t:t+max_h]

            errors[name].extend(np.abs(actual - forecast))
        except Exception:
            pass

# Print final MAE results
print("Time Series Cross-Validation MAE Results (averaged over horizons 1-12):")
for name in models_config:
    mae = np.mean(errors[name])
    print(f" {name}: MAE = {mae:.6f}")

```

Time Series Cross-Validation MAE Results (averaged over horizons 1-12):

```

ARIMA(2,1,0): MAE = 0.666044
ARIMA(0,1,2): MAE = 0.593214
ARIMA(2,1,0)(1,1,0)12: MAE = 0.100515
ARIMA(0,1,2)(0,1,1)12: MAE = 0.111516

```

15. (Bonus) Break down the MAE for the forecasts made in Q14 by forecast horizon. That is, for each $h = 1, \dots, 12$, calculate the MAE of the h -step ahead forecasts made by each model. Make a plot with the horizon h on the x-axis and MAE on the y-axis, and compare in particular the models $\text{ARIMA}(2, 1, 0)(1, 1, 0)_{12}$ and $\text{ARIMA}(0, 1, 2)(0, 1, 1)_{12}$. Do you see anything interesting happening here in the comparison between their MAE as we vary the horizon h ?

Answer.

(Type your answer and discussion here)

```
# Plot MAE by forecast horizon here
```

16. (Bonus²) Evaluate the forecasts made by auto-ARIMA in this time series CV pipeline. Remember, this means that auto-ARIMA will be rerun (yikes!) at each iteration in time series CV. This may take a very long time to run (which is why this is a Bonus²). If it finishes for you, how does its MAE compare?

Answer.

(Type your answer and discussion here)

```
# Run auto-ARIMA and evaluate MAE here
```